Relatório Projeto 3 V1.0 AED 2024/2025

Nome: Francisco Carvalho Gouveia
Nº Estudante: 2023214517
PL (inscrição): PL4 Email: uc2023214517@student.uc.pt

IMPORTANTE:

- **Os textos das conclusões devem ser manuscritos...** o texto deve obedecer a este requisito para não ser penalizado.
- Texto para além das linhas reservadas, ou que não seja legível para um leitor comum, não será tido em conta.
- **O relatório deve ser submetido num único PDF** que deve incluir os anexos. A não observância deste formato é penalizada.

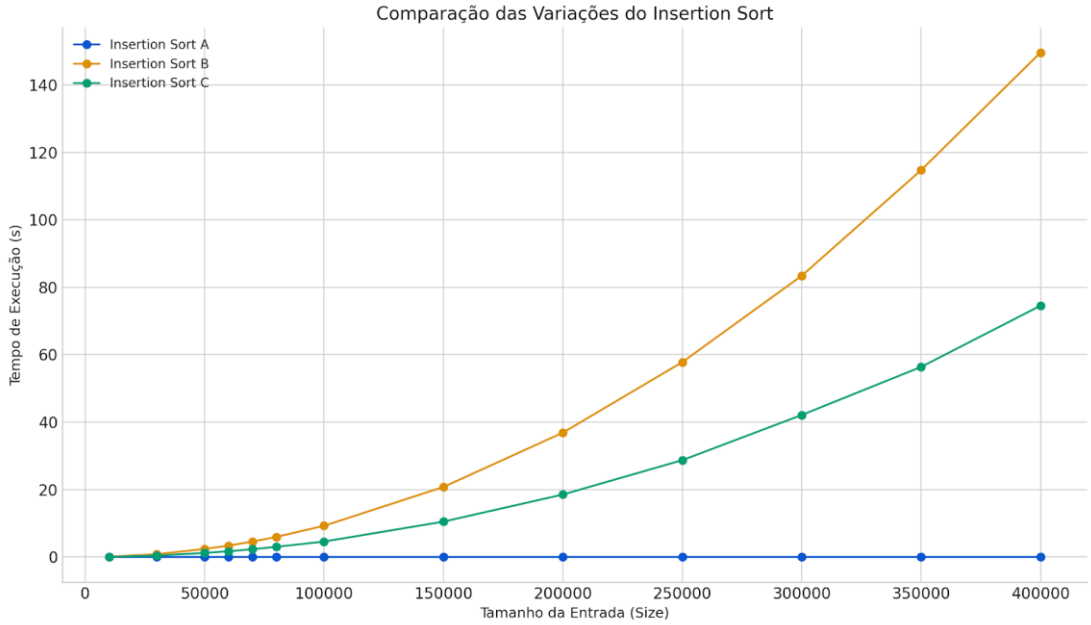
1. Planeamento

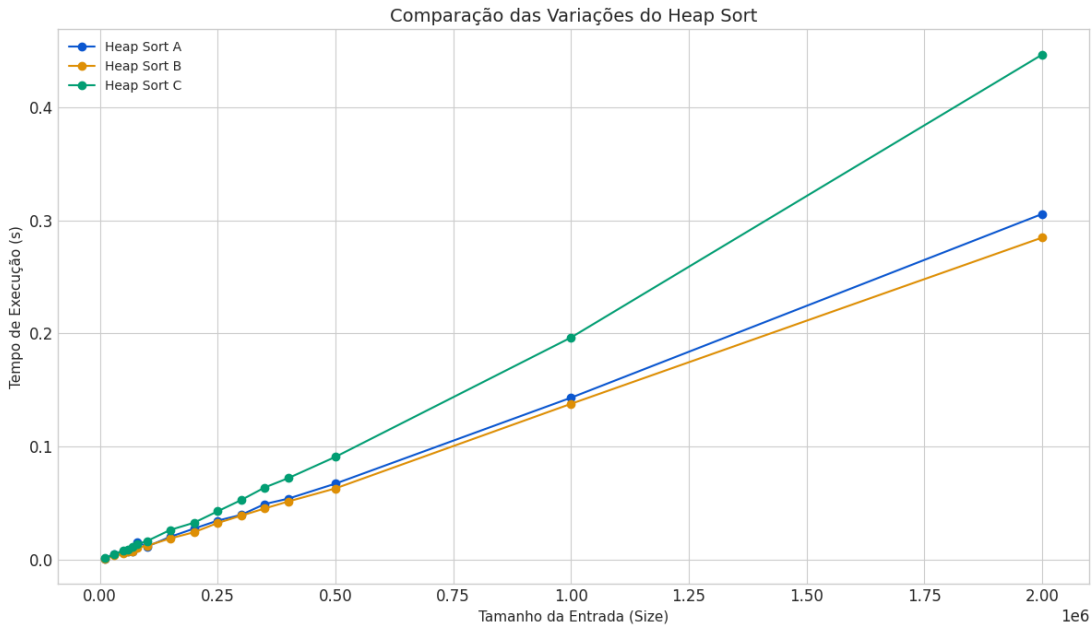
	Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	...
T1: Insertion Sort	**Ver implementação**	**Implementação**				
T2: Heap Sort		**Ver implementação**	**implementação**			
T3: Quick Sort			**Ver implementação**	**implementação**		
Finalização Relatório	**Iniciar**				**Finalização**	

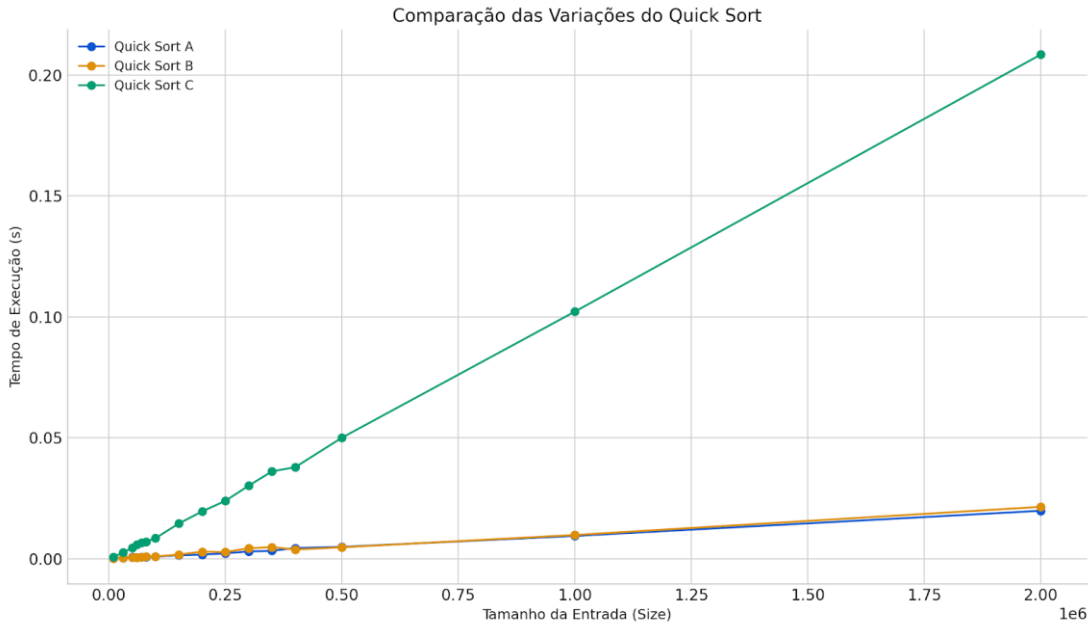
2. Recolha de Resultados *(tabelas)*

Nome do Algoritmo	10000	30000	50000	60000	70000	80000	100000	150000	200000	250000	300000	350000	400000	500000	1000000	2000000
Insertion Sort A	0,00003	0,000031	0,00005	0,000075	0,000076	0,000105	0,000104	0,000168	0,000206	0,000249	0,000312	0,000395	0,00048	N/A	N/A	N/A
Insertion Sort B	0,101549	0,842792	2,394169	3,384828	4,57476	5,966681	9,253847	20,7724	36,87795	57,79637	83,40932	114,7334	149,6017	N/A	N/A	N/A
Insertion Sort C	0,056543	0,421043	1,230412	1,706628	2,30181	3,026128	4,580266	10,49614	18,53413	28,71747	42,12038	56,3798	74,5441	N/A	N/A	N/A
Heap Sort A	0,000971	0,004688	0,006528	0,007211	0,007293	0,015549	0,011475	0,020212	0,027385	0,034448	0,039533	0,049103	0,053897	0,067154	0,143174	0,305707
Heap Sort B	0,000954	0,003522	0,005293	0,006898	0,007419	0,010654	0,012119	0,018771	0,024178	0,032353	0,038826	0,04521	0,051401	0,062942	0,137769	0,284834
Heap Sort C	0,001299	0,004383	0,00838	0,008731	0,011124	0,013896	0,016323	0,026301	0,032583	0,042685	0,052648	0,063767	0,072047	0,090826	0,196428	0,446718
Quick Sort A	0,000087	0,00025	0,000649	0,000497	0,000738	0,000688	0,000815	0,001428	0,001663	0,002207	0,002986	0,003276	0,004388	0,004888	0,009398	0,01981
Quick Sort B	0,000081	0,000273	0,000475	0,000521	0,000719	0,000934	0,000915	0,001712	0,002887	0,002749	0,004298	0,004789	0,003778	0,004703	0,009795	0,021434
Quick Sort C	0,000741	0,002536	0,004442	0,005853	0,006593	0,007051	0,008468	0,014534	0,019526	0,023919	0,030172	0,036091	0,037833	0,050017	0,102156	0,208454

3. Visualização de Resultados *(gráficos)*







4. Conclusões *(as linhas no template representam a extensão máxima de texto manuscrito)*

4.1 Tarefa 1

O Insertion Sort é um algoritmo simples de ordenação que percorre o vetor da esquerda para a direita, inserindo cada elemento na posição correta em relação aos anteriores.

- Chave A: Como os elementos estão na ordem correta, não são feitas trocas, apenas comparações mínimas. O tempo cresce linearmente $O(n)$
- Chave B: Cada novo elemento precisa ser comparado e movido até ao início do vetor exigindo muitas trocas $O(n^2)$
- Chave C: Há trocas e comparações, mas em menor escala do que no pior caso, $O(n)$ ou $O(n^2)$

Heap Sort utiliza uma estrutura de dados chamada heap (geralmente um max-heap) para ordenar os elementos. Ele primeiro transforma o vetor em um heap, depois extrai o maior elemento repetidamente, reorganizando o heap a cada extração. Independentemente da ordem inicial dos dados, o algoritmo mantém desempenho

4.2 Tarefa 2

Heap Sort utiliza uma estrutura de dados chamada (geralmente um max-heap) para ordenar os elementos. Ele transforma o vetor em um heap, depois extrai o maior repetidamente, reorganizando o heap a cada extração. Inde da ordem inicial dos dados, o algoritmo mantem dese consistente O(nlogn)

4.3 Tarefa 3

Quick Sort é um algoritmo de divisão e conquista que escolhe um pivô, particiona o vetor em elementos menores e maiores que ele, e aplica o mesmo processo recursivamente às sublistas.

- Melhor caso (bom pivô): As divisões são equilibradas e o desempenho é excelente O(nlogn)
- Pior caso (pivô mau): as divisões ficam muito desiquilibradas degradando o desempenho $O(n^2)$

Anexo B - Código de Autor

```
#include <stdio.h>

#include <stdlib.h> // Necessário para malloc e free

#include <time.h>


#include "keys.c"
```

```
#define TIMES 3

void create_arrays(int *arr_A, int *arr_B, int *arr_C, int size){
   keys_A(arr_A, size);
   keys_B(arr_B, size);
   keys_C(arr_C, size);
}

void insertion_sort(int *arr,  int SIZE){
   int temp;
   for(int i = 1; i < SIZE; i++){
      for(int j = i - 1; j >= 0; j--){
         if(arr[j + 1] < arr[j]){
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
         }else{
            break;
         }
      }
   }
}

void max_heap(int i, int *arr, int SIZE){
   int left_child = 2 * i + 1;
   int right_child = 2 * i + 2;
   int largest = i;

   if(left_child < SIZE && arr[left_child] > arr[largest]){
      largest = left_child;
   }
   if(right_child < SIZE && arr[right_child] > arr[largest]){
      largest = right_child;
   }
   if(largest != i){
```

```
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
        max_heap(largest, arr, SIZE);
    }
}

void heap_sort(int *arr, int SIZE){

    for(int i = SIZE / 2 - 1; i >= 0; i--){
        max_heap(i, arr, SIZE);
    }

    for(int i = SIZE - 1; i > 0; i--){
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;
        max_heap(0, arr, i);
    }

}

void swap(int *a, int *b){
    int temp = *a;
    *a = *b;
    *b = temp;
}

#define QUICK_SORT_CUTOFF 30

void quick_sort(int *arr, int size){
    if (size <= 1) {
        return;
    }
```

```
if (size <= QUICK_SORT_CUTOFF) {
    insertion_sort(arr, size);
    return;
}

int pivot_val;
int i = 0;
int j = size - 1;

if (size > 2) {
    int mid_idx = (size - 1) / 2; // Middle index for arr[0...size-1]

    // Sort arr[0], arr[mid_idx], arr[size-1] to find the median
    if (arr[0] > arr[mid_idx]) {
        swap(&arr[0], &arr[mid_idx]);
    }
    if (arr[0] > arr[size - 1]) {
        swap(&arr[0], &arr[size - 1]);
    }
    if (arr[mid_idx] > arr[size - 1]) {
        swap(&arr[mid_idx], &arr[size - 1]);
    }
    // Now arr[mid_idx] holds the median of the three values.
    pivot_val = arr[mid_idx];
} else {
    // Fallback for size = 2 (CUTOFF usually handles this, but as a safeguard)
    pivot_val = arr[size / 2];
}

// Hoare Partitioning (original structure)
// i and j are already initialized to 0 and size-1
int current_i = i;
int current_j = j;
```

```
    while(current_i <= current_j){
        // 3. Efficient handling of elements equal to pivot (inherent in Hoare partition)
        while(arr[current_i] < pivot_val){
            current_i++;
        }

        while(arr[current_j] > pivot_val){
            current_j--;
        }

        if(current_i <= current_j){
            swap(&arr[current_i], &arr[current_j]);
            current_i++;
            current_j--;
        }
    }

    // Recursive calls on the two partitions
    // Left partition: arr[0...current_j]
    // Size of left partition: current_j + 1
    if (current_j > 0) { // If there's a left partition with at least one element (original was j > 0)
        quick_sort(arr, current_j + 1);
    }

    // Right partition: arr[current_i...size-1]
    // Starts at arr + current_i, size is size - current_i
    if (current_i < size) { // If there's a right partition
        quick_sort(arr + current_i, size - current_i);
    }

}

int main(){
```

```
int values[] = {10000,30000, 50000, 60000, 70000, 80000, 100000, 150000, 200000, 250000, 300000,
350000, 400000, 500000, 1000000};
srand(time(NULL));
clock_t start;

for(int l = 0; l < TIMES; l++){

    printf("--- Iteration %d ---\n", l + 1);
    for(int k = 0; k < (int)(sizeof(values)/sizeof(int)) ; k++){

        printf("Size: %d\n", values[k]);

        // Alocar arrays dinamicamente na heap
        int *arr_A = (int*)malloc(values[k] * sizeof(int));
        int *arr_B = (int*)malloc(values[k] * sizeof(int));
        int *arr_C = (int*)malloc(values[k] * sizeof(int));

        // Verificar se a alocação foi bem-sucedida
        if (arr_A == NULL || arr_B == NULL || arr_C == NULL) {
            fprintf(stderr, "Memory allocation failed for size %d\n", values[k]);
            // Liberar memória que possa ter sido alocada com sucesso antes de continuar/sair
            if (arr_A != NULL) free(arr_A);
            if (arr_B != NULL) free(arr_B);
            if (arr_C != NULL) free(arr_C);
            continue; // Pula para o próximo tamanho de array ou próxima iteração de TIMES
        }

        create_arrays(arr_A, arr_B, arr_C, values[k]);

        if (k < 10){ // Limita o insertion_sort para tamanhos menores
            // Teste Insertion Sort para arr_A
            create_arrays(arr_A, arr_B, arr_C, values[k]); // Garante dados frescos para arr_A
            start = clock();
            insertion_sort(arr_A, values[k]);
            printf("Insertion Sort A: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);
```

```
    // Teste Insertion Sort para arr_B
    create_arrays(arr_A, arr_B, arr_C, values[k]); // Garante dados frescos para arr_B
    start = clock();
    insertion_sort(arr_B, values[k]);
    printf("Insertion Sort B: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);

    // Teste Insertion Sort para arr_C
    create_arrays(arr_A, arr_B, arr_C, values[k]); // Garante dados frescos para arr_C
    start = clock();
    insertion_sort(arr_C, values[k]);
    printf("Insertion Sort C: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);
    printf("\n");
}

// Teste Heap Sort
create_arrays(arr_A, arr_B, arr_C, values[k]);
start = clock();
heap_sort(arr_A, values[k]);
printf("Heap Sort A: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);

create_arrays(arr_A, arr_B, arr_C, values[k]);
start = clock();
heap_sort(arr_B, values[k]);
printf("Heap Sort B: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);

create_arrays(arr_A, arr_B, arr_C, values[k]);
start = clock();
heap_sort(arr_C, values[k]);
printf("Heap Sort C: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);
printf("\n");

// Teste Quick Sort
create_arrays(arr_A, arr_B, arr_C, values[k]);
start = clock();
```

```
        quick_sort(arr_A, values[k]);
        printf("Quick Sort A: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);

        create_arrays(arr_A, arr_B, arr_C, values[k]);
        start = clock();
        quick_sort(arr_B, values[k]);
        printf("Quick Sort B: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);

        create_arrays(arr_A, arr_B, arr_C, values[k]);
        start = clock();
        quick_sort(arr_C, values[k]);
        printf("Quick Sort C: %f s\n", (double)(clock() - start) / CLOCKS_PER_SEC);
        printf("\n\n");

        // Liberar a memória alocada para evitar vazamentos de memória
        free(arr_A);
        free(arr_B);
        free(arr_C);
      }
    }
    return 0;
}
```

Anexo C - Referências

https://www.geeksforgeeks.org/quick-sort-algorithm/

https://www.geeksforgeeks.org/heap-sort/